DATA STRUCTURES AND ALGORITHMS

Practical Record Companion

Algorithms • Flowcharts • C Programs

Prepared in the exact order of the laboratory assignments in the supplied syllabus.

Includes Practicals 1–10 and the two items listed under “Content Beyond Syllabus” (11–12).

Contents

1. Linear Search and Binary Search

2. Sort Student Roll Numbers

3. Sparse Matrix Operations

4. Singly Linked List Operations

5. Polynomial ADT

6. Linked-List Stack and Postfix Evaluation

7. Evaluate an Infix Expression

8. Tower of Hanoi Using Recursion

9. Double-Ended Queue (Deque)

10. Binary Search Tree Operations

11. Build a Max Heap

12. Hash Table Using Hashing

Notes

Language: C (standard C11 style). The syllabus gives the problem statements, not prescribed algorithms or code; the implementations below are reference solutions based on those statements. Flowcharts are text flowcharts for compact printing. Compile each program separately. For Practical 1, enter an array sorted in ascending order for binary search.

Practical 1: Linear Search and Binary Search

Problem statement

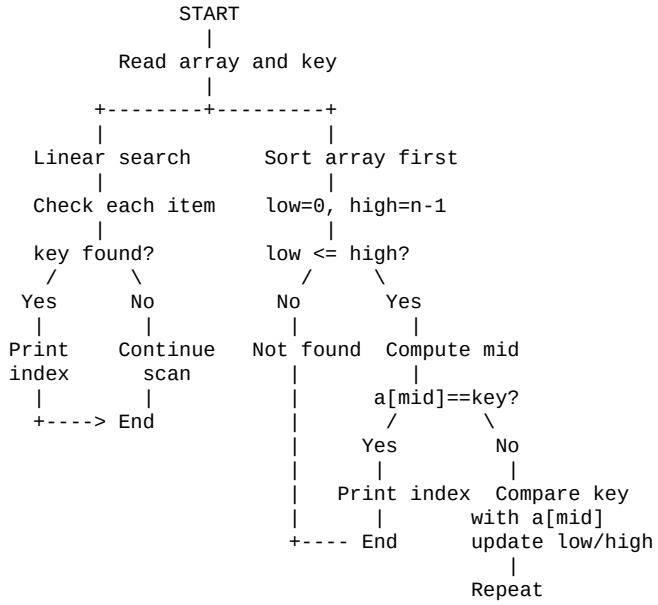
Implement linear search and binary search for a given array.

Algorithm

```
LINEAR SEARCH
1. Read n, the array, and the key.
2. Scan each element from index 0 to n-1.
3. If an element equals key, report its position and stop.
4. If the scan ends, report that the key was not found.

BINARY SEARCH
1. Ensure the array is sorted in ascending order.
2. Set low = 0 and high = n-1.
3. While low <= high, calculate mid = low + (high-low)/2.
4. If a[mid] equals key, report its position.
5. If key is smaller, set high = mid-1; otherwise set low = mid+1.
6. If the loop ends, report not found.
```

Flowchart



C Program

```c
#include <stdio.h>

int linearSearch(int a[], int n, int key) {
    for (int i = 0; i < n; i++)
        if (a[i] == key) return i;
    return -1;
}

int binarySearch(int a[], int n, int key) {
    int low = 0, high = n - 1;
    while (low <= high) {
        int mid = low + (high - low) / 2;
        if (a[mid] == key) return mid;
        if (a[mid] < key) low = mid + 1;
        else high = mid - 1;
    }
    return -1;
}

int main(void) {
    int a[100], n, key, result;
    printf("Enter number of elements: ");
    scanf("%d", &n);
    if (n < 1 || n > 100) return 1;
    printf("Enter elements in ascending order for binary search:\n");
    for (int i = 0; i < n; i++) scanf("%d", &a[i]);
    printf("Enter key: ");
    scanf("%d", &key);

    result = linearSearch(a, n, key);
```

```
    if (result >= 0) printf("Linear search: found at position %d\n", result + 1);
    else printf("Linear search: not found\n");

    result = binarySearch(a, n, key);
    if (result >= 0) printf("Binary search: found at position %d\n", result + 1);
    else printf("Binary search: not found\n");
    return 0;
}
```

Practical 2: Sort Student Roll Numbers

Problem statement

Arrange student roll numbers in ascending order using (a) Bubble Sort, (b) Insertion Sort, and (c) Quick Sort.

Algorithm

```
BUBBLE SORT
1. For each pass, compare adjacent elements.
2. Swap them if they are in the wrong order.
3. Stop after n-1 passes or when a pass makes no swaps.

INSERTION SORT
1. Starting at index 1, save the current value as key.
2. Shift earlier values greater than key one position right.
3. Insert key in the open position.

QUICK SORT
1. Choose the last element as pivot.
2. Move elements <= pivot to its left and larger elements to its right.
3. Place pivot in its final position.
4. Recursively sort the left and right partitions.
```

Flowchart

```
 START
   |
Read roll numbers
   |
Choose sorting method
   |
+-- Bubble: compare neighbours, swap if needed,
|           repeat passes until sorted
|
+-- Insertion: take next item, shift larger
|              items right, insert item
|
+-- Quick: choose pivot, partition array,
|          recursively sort both sides
   |
Display ascending roll numbers
   |
  END
```

C Program

```c
#include <stdio.h>

void printArray(int a[], int n) {
    for (int i = 0; i < n; i++) printf("%d ", a[i]);
    printf("\n");
}
void bubbleSort(int a[], int n) {
    for (int i = 0; i < n - 1; i++) {
        int swapped = 0;
        for (int j = 0; j < n - 1 - i; j++)
            if (a[j] > a[j + 1]) {
                int t = a[j]; a[j] = a[j + 1]; a[j + 1] = t;
                swapped = 1;
            }
        if (!swapped) break;
    }
}
void insertionSort(int a[], int n) {
    for (int i = 1; i < n; i++) {
        int key = a[i], j = i - 1;
        while (j >= 0 && a[j] > key) { a[j + 1] = a[j]; j--; }
        a[j + 1] = key;
    }
}
int partition(int a[], int low, int high) {
    int pivot = a[high], i = low - 1;
    for (int j = low; j < high; j++)
        if (a[j] <= pivot) {
            i++;
            int t = a[i]; a[i] = a[j]; a[j] = t;
        }
    int t = a[i + 1]; a[i + 1] = a[high]; a[high] = t;
    return i + 1;
}
void quickSort(int a[], int low, int high) {
```

```
    if (low < high) {
        int p = partition(a, low, high);
        quickSort(a, low, p - 1);
        quickSort(a, p + 1, high);
    }
}
int main(void) {
    int n, original[100], a[100];
    printf("Enter number of students: ");
    scanf("%d", &n);
    if (n < 1 || n > 100) return 1;
    printf("Enter roll numbers: ");
    for (int i = 0; i < n; i++) scanf("%d", &original[i]);

    for (int i = 0; i < n; i++) a[i] = original[i];
    bubbleSort(a, n); printf("Bubble sort: "); printArray(a, n);

    for (int i = 0; i < n; i++) a[i] = original[i];
    insertionSort(a, n); printf("Insertion sort: "); printArray(a, n);

    for (int i = 0; i < n; i++) a[i] = original[i];
    quickSort(a, 0, n - 1); printf("Quick sort: "); printArray(a, n);
    return 0;
}
```

Practical 3: Sparse Matrix Operations

Problem statement

Represent a sparse matrix and support initialization, insertion, row-column sorting, and transpose.

Algorithm

```
1. Store only non-zero values as triples (row, column, value), using 0-based indices.
2. Initialize the sparse matrix with zero stored entries.
3. To insert, validate the coordinates. Replace an existing entry at the same coordinate, remove it if value is zero, or app
4. Sort entries by row and then column.
5. To transpose, swap row and column for every entry, swap total row and column counts, then sort again.
6. Display dimensions and stored triples.
```

Flowchart

```
 START
   |
Read matrix dimensions
   |
Initialize empty triple list
   |
Choose operation
   |
+-- Insert/update non-zero entry
+-- Sort triples by row, then column
+-- Transpose: swap row and column
+-- Display dimensions and triples
   |
More operations? -- Yes --> Choose operation
   |
   No
   |
  END
```

C Program

```c
#include <stdio.h>
#define MAX 500

typedef struct { int row, col, value; } Term;
typedef struct {
    int rows, cols, count;
    Term t[MAX];
} Sparse;

void sortTerms(Sparse *s) {
    for (int i = 0; i < s->count - 1; i++)
        for (int j = 0; j < s->count - 1 - i; j++)
            if (s->t[j].row > s->t[j+1].row ||
               (s->t[j].row == s->t[j+1].row &&
                s->t[j].col > s->t[j+1].col)) {
                Term x = s->t[j]; s->t[j] = s->t[j+1]; s->t[j+1] = x;
            }
}
void insertTerm(Sparse *s, int r, int c, int v) {
    if (r < 0 || r >= s->rows || c < 0 || c >= s->cols) {
        printf("Invalid position.\n"); return;
    }
    for (int i = 0; i < s->count; i++)
        if (s->t[i].row == r && s->t[i].col == c) {
            if (v == 0) {
                for (int j = i; j < s->count - 1; j++) s->t[j] = s->t[j+1];
                s->count--;
            } else s->t[i].value = v;
            sortTerms(s); return;
        }
    if (v == 0) return;
    if (s->count == MAX) { printf("Storage full.\n"); return; }
    s->t[s->count++] = (Term){r, c, v};
    sortTerms(s);
}
void transpose(Sparse *s) {
    for (int i = 0; i < s->count; i++) {
        int x = s->t[i].row; s->t[i].row = s->t[i].col; s->t[i].col = x;
    }
    int x = s->rows; s->rows = s->cols; s->cols = x;
    sortTerms(s);
}
void display(Sparse *s) {
    printf("Dimensions: %d x %d; non-zero entries: %d\n", s->rows, s->cols, s->count);
```

```
    printf("Row Col Value\n");
    for (int i = 0; i < s->count; i++)
        printf("%3d %3d %5d\n", s->t[i].row, s->t[i].col, s->t[i].value);
}
int main(void) {
    Sparse s = {0};
    int choice, r, c, v;
    printf("Enter rows and columns: ");
    if (scanf("%d%d", &s.rows, &s.cols) != 2 || s.rows <= 0 || s.cols <= 0) return 1;
    do {
        printf("\n1.Insert/update 2.Sort 3.Transpose 4.Display 0.Exit: ");
        scanf("%d", &choice);
        switch (choice) {
            case 1: printf("Enter row column value (0-based): "); scanf("%d%d%d", &r,&c,&v); insertTerm(&s,r,c,v); break;
            case 2: sortTerms(&s); printf("Sorted.\n"); break;
            case 3: transpose(&s); printf("Transposed.\n"); break;
            case 4: display(&s); break;
            case 0: break;
            default: printf("Invalid choice.\n");
        }
    } while (choice != 0);
    return 0;
}
```

Practical 4: Singly Linked List Operations

Problem statement

Write functions to add and delete nodes, count nodes, and display the list in reverse order using recursion.

Algorithm

1. Maintain a head pointer, initially NULL.
2. Insert: allocate a node and attach it at the end of the list.
3. Delete: locate the first node containing the requested value, unlink it, and free it.
4. Count: traverse from head to NULL and increment a counter.
5. Reverse display: recursively visit next nodes first, then print the current node while recursion unwinds.
6. Repeat operations until the user exits.

Flowchart

```
 START
   |
head = NULL
   |
Choose operation
   |
+-- Add: allocate node, link at end
+-- Delete: locate node, unlink and free
+-- Count: traverse and increment counter
+-- Reverse display: recurse to end, print on return
   |
More operations? -- Yes --> Choose operation
   |
   No
   |
Free remaining nodes
   |
  END
```

C Program

```c
#include <stdio.h>
#include <stdlib.h>

typedef struct Node { int data; struct Node *next; } Node;

void insertEnd(Node **head, int value) {
    Node *n = malloc(sizeof *n);
    if (!n) { printf("Memory allocation failed.\n"); return; }
    n->data = value; n->next = NULL;
    if (!*head) { *head = n; return; }
    Node *p = *head;
    while (p->next) p = p->next;
    p->next = n;
}
int deleteValue(Node **head, int value) {
    Node *p = *head, *prev = NULL;
    while (p && p->data != value) { prev = p; p = p->next; }
    if (!p) return 0;
    if (prev) prev->next = p->next; else *head = p->next;
    free(p); return 1;
}
int countNodes(Node *head) {
    int count = 0;
    for (Node *p = head; p; p = p->next) count++;
    return count;
}
void reverseDisplay(Node *head) {
    if (!head) return;
    reverseDisplay(head->next);
    printf("%d ", head->data);
}
void display(Node *head) {
    for (Node *p = head; p; p = p->next) printf("%d -> ", p->data);
    printf("NULL\n");
}
void freeList(Node *head) {
    while (head) { Node *next = head->next; free(head); head = next; }
}
int main(void) {
    Node *head = NULL;
    int ch, value;
    do {
        printf("\n1.Add 2.Delete by value 3.Count 4.Reverse display 5.Display 0.Exit: ");
        scanf("%d", &ch);
```

```
        switch (ch) {
            case 1: printf("Value: "); scanf("%d", &value); insertEnd(&head, value); break;
            case 2: printf("Value to delete: "); scanf("%d", &value);
                    printf(deleteValue(&head, value) ? "Deleted.\n" : "Value not found.\n"); break;
            case 3: printf("Node count: %d\n", countNodes(head)); break;
            case 4: reverseDisplay(head); printf("\n"); break;
            case 5: display(head); break;
            case 0: break;
            default: printf("Invalid choice.\n");
        }
    } while (ch != 0);
    freeList(head);
    return 0;
}
```

Practical 5: Polynomial ADT

Problem statement

Represent a polynomial and support creating an empty polynomial, inserting terms, adding two polynomials, and evaluating a polynomial.

Algorithm

```
1. Represent each term by coefficient and exponent.
2. Insert a term: if its exponent already exists, add coefficients; otherwise add a new term. Remove terms whose resulting c
3. To add polynomials, copy terms from the first polynomial, then insert terms from the second so equal exponents combine.
4. To evaluate at x, compute the sum of coefficient * x^exponent for every term.
5. Display terms in descending exponent order.
```

Flowchart

```
 START
   |
Create empty polynomial P and Q
   |
Insert terms into P and Q
   |
Choose operation
   |
+-- Add: merge terms with equal exponents
+-- Evaluate: sum coefficient * x^exponent
+-- Display polynomial terms
   |
Display result
   |
  END
```

C Program

```c
#include <stdio.h>
#include <math.h>
#define MAX 100

typedef struct { double coeff; int exp; } Term;
typedef struct { Term t[MAX]; int n; } Poly;

void insert(Poly *p, double c, int e) {
    if (c == 0) return;
    for (int i = 0; i < p->n; i++)
        if (p->t[i].exp == e) {
            p->t[i].coeff += c;
            if (fabs(p->t[i].coeff) < 1e-9) {
                for (int j = i; j < p->n - 1; j++) p->t[j] = p->t[j+1];
                p->n--;
            }
            return;
        }
    if (p->n == MAX) { printf("Polynomial full.\n"); return; }
    p->t[p->n++] = (Term){c, e};
}
void sortPoly(Poly *p) {
    for (int i = 0; i < p->n - 1; i++)
        for (int j = 0; j < p->n - 1 - i; j++)
            if (p->t[j].exp < p->t[j+1].exp) {
                Term x = p->t[j]; p->t[j] = p->t[j+1]; p->t[j+1] = x;
            }
}
Poly add(Poly a, Poly b) {
    Poly r = {0};
    for (int i = 0; i < a.n; i++) insert(&r, a.t[i].coeff, a.t[i].exp);
    for (int i = 0; i < b.n; i++) insert(&r, b.t[i].coeff, b.t[i].exp);
    sortPoly(&r); return r;
}
double evaluate(Poly p, double x) {
    double sum = 0;
    for (int i = 0; i < p.n; i++) sum += p.t[i].coeff * pow(x, p.t[i].exp);
    return sum;
}
void readPoly(Poly *p) {
    int n, e; double c;
    printf("Number of terms: "); scanf("%d", &n);
    for (int i = 0; i < n; i++) {
        printf("Coefficient and non-negative exponent: ");
        scanf("%lf%d", &c, &e);
        if (e < 0) { printf("Negative exponent skipped.\n"); continue; }
```

```
        insert(p, c, e);
    }
    sortPoly(p);
}
void display(Poly p) {
    if (!p.n) { printf("0\n"); return; }
    for (int i = 0; i < p.n; i++) {
        if (i) printf(" + ");
        printf("%.2gx^%d", p.t[i].coeff, p.t[i].exp);
    }
    printf("\n");
}
int main(void) {
    Poly p = {0}, q = {0};
    printf("Polynomial P:\n"); readPoly(&p);
    printf("Polynomial Q:\n"); readPoly(&q);
    printf("P = "); display(p);
    printf("Q = "); display(q);
    Poly r = add(p, q);
    printf("P + Q = "); display(r);
    double x; printf("Enter x to evaluate P: "); scanf("%lf", &x);
    printf("P(%.3g) = %.6g\n", x, evaluate(p, x));
    return 0;
}
```

Practical 6: Linked-List Stack and Postfix Evaluation

Problem statement

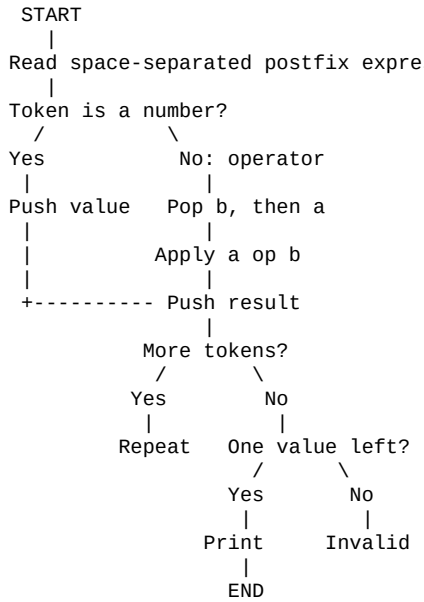
Implement a stack using a linked list and use it to evaluate a postfix expression.

Algorithm

```
STACK OPERATIONS
1. Push: allocate a node and place it at the top.
2. Pop: remove and return the top node; report underflow if empty.

POSTFIX EVALUATION (space-separated integer tokens)
1. Scan tokens from left to right.
2. If a token is an integer, push it.
3. If it is an operator, pop right operand b, then left operand a.
4. Compute a operator b and push the result.
5. After all tokens, exactly one value should remain; that is the answer.
6. Reject malformed expressions and division by zero.
```

Flowchart



C Program

```c
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <ctype.h>

typedef struct Node { long long value; struct Node *next; } Node;
int push(Node **top, long long v) {
    Node *n = malloc(sizeof *n);
    if (!n) return 0;
    n->value = v; n->next = *top; *top = n; return 1;
}
int pop(Node **top, long long *v) {
    if (!*top) return 0;
    Node *n = *top; *v = n->value; *top = n->next; free(n); return 1;
}
void clearStack(Node **top) {
    long long v; while (pop(top, &v)) {}
}
int main(void) {
    char line[1024], *tok;
    Node *top = NULL;
    printf("Enter postfix expression with spaces (example: 12 3 + 4 *):\n");
    if (!fgets(line, sizeof line, stdin)) return 1;
    tok = strtok(line, " \t\r\n");
    while (tok) {
        char *end;
        long long value = strtoll(tok, &end, 10);
        if (*tok && *end == '\0') {
            if (!push(&top, value)) { clearStack(&top); return 1; }
        } else if (strlen(tok) == 1 && strchr("+-*/%", tok[0])) {
            long long a, b, result;
```

```
            if (!pop(&top, &b) || !pop(&top, &a)) {
                printf("Invalid postfix expression.\n"); clearStack(&top); return 1;
            }
            switch (tok[0]) {
                case '+': result = a + b; break;
                case '-': result = a - b; break;
                case '*': result = a * b; break;
                case '/': if (b == 0) { printf("Division by zero.\n"); clearStack(&top); return 1; } result = a / b; break;
                default: if (b == 0) { printf("Modulo by zero.\n"); clearStack(&top); return 1; } result = a % b;
            }
            if (!push(&top, result)) { clearStack(&top); return 1; }
        } else {
            printf("Invalid token: %s\n", tok); clearStack(&top); return 1;
        }
        tok = strtok(NULL, " \t\r\n");
    }
    long long answer;
    if (!pop(&top, &answer) || top) {
        printf("Invalid postfix expression.\n"); clearStack(&top); return 1;
    }
    printf("Result = %lld\n", answer);
    return 0;
}
```

Practical 7: Evaluate an Infix Expression

Problem statement

Evaluate an infix expression directly, handling spaces, parentheses, and operator precedence without converting it to postfix.

Algorithm

```
1. Ignore whitespace and parse integers (including multi-digit values).
2. Use one stack for values and one stack for operators.
3. Push '(' onto the operator stack.
4. On ')', apply operators until '(' is found, then discard '('.
5. For an operator, apply operators on the stack with greater or equal precedence, then push the new operator.
6. At end of input, apply remaining operators.
7. Check for missing operands, mismatched parentheses, and division by zero.
```

Flowchart

```
 START
   |
Read infix expression
   |
Scan next non-space token
   |
Number? -> Push value
   |
'('? -> Push operator
')'? -> Apply until '('; discard '('
Operator? -> Apply higher/equal precedence operators,
             then push current operator
   |
More input? -- Yes --> Scan next token
   |
   No
   |
Apply remaining operators
   |
Exactly one value and no errors?
       /          \
     Yes           No
      |             |
 Print result    Print error
      |
     END
```

C Program

```c
#include <stdio.h>
#include <stdlib.h>
#include <ctype.h>
#include <string.h>

#define MAX 512
long long values[MAX]; char ops[MAX]; int nv = 0, no = 0;
int precedence(char op) { return (op == '+' || op == '-') ? 1 : (op == '*' || op == '/' || op == '%') ? 2 : 0; }
int apply(void) {
    if (no == 0 || nv < 2) return 0;
    char op = ops[--no]; long long b = values[--nv], a = values[--nv], r;
    switch (op) {
        case '+': r = a + b; break; case '-': r = a - b; break;
        case '*': r = a * b; break;
        case '/': if (!b) return 0; r = a / b; break;
        case '%': if (!b) return 0; r = a % b; break;
        default: return 0;
    }
    values[nv++] = r; return 1;
}
int main(void) {
    char s[1024]; int expectValue = 1;
    printf("Enter infix expression (integer arithmetic, + - * / %% and parentheses):\n");
    if (!fgets(s, sizeof s, stdin)) return 1;
    for (int i = 0; s[i]; ) {
        if (isspace((unsigned char)s[i])) { i++; continue; }
        if (isdigit((unsigned char)s[i])) {
            if (!expectValue || nv >= MAX) { printf("Invalid expression.\n"); return 1; }
            long long v = 0;
            while (isdigit((unsigned char)s[i])) { v = v * 10 + (s[i] - '0'); i++; }
            values[nv++] = v; expectValue = 0;
        } else if (s[i] == '(') {
            if (!expectValue || no >= MAX) { printf("Invalid expression.\n"); return 1; }
```

```
            ops[no++] = s[i++]; expectValue = 1;
        } else if (s[i] == ')') {
            if (expectValue) { printf("Invalid expression.\n"); return 1; }
            while (no && ops[no-1] != '(') if (!apply()) { printf("Evaluation error.\n"); return 1; }
            if (!no) { printf("Mismatched parentheses.\n"); return 1; }
            no--; i++; expectValue = 0;
        } else if (strchr("+-*/%", s[i])) {
            char op = s[i++];
            if (expectValue || no >= MAX) { printf("Invalid expression.\n"); return 1; }
            while (no && ops[no-1] != '(' && precedence(ops[no-1]) >= precedence(op))
                if (!apply()) { printf("Division by zero or invalid expression.\n"); return 1; }
            ops[no++] = op; expectValue = 1;
        } else { printf("Unsupported character: %c\n", s[i]); return 1; }
    }
    if (expectValue) { printf("Incomplete expression.\n"); return 1; }
    while (no) {
        if (ops[no-1] == '(' || !apply()) { printf("Invalid expression or division by zero.\n"); return 1; }
    }
    if (nv != 1) { printf("Invalid expression.\n"); return 1; }
    printf("Result = %lld\n", values[0]);
    return 0;
}
```

Practical 8: Tower of Hanoi Using Recursion

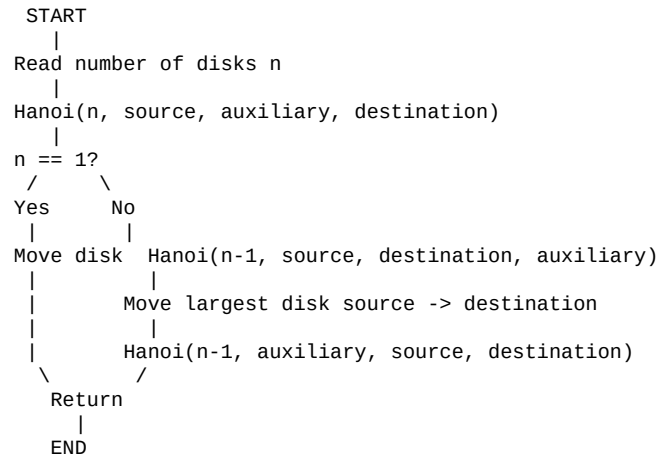
Problem statement

Implement the Tower of Hanoi problem using recursion.

Algorithm

1. If n = 1, move the disk from source to destination.
2. Recursively move n-1 disks from source to auxiliary using destination.
3. Move the largest disk from source to destination.
4. Recursively move n-1 disks from auxiliary to destination using source.
5. The minimum number of moves is 2^n - 1.

Flowchart



C Program

```c
#include <stdio.h>

void hanoi(int n, char source, char auxiliary, char destination) {
    if (n == 1) {
        printf("Move disk 1 from %c to %c\n", source, destination);
        return;
    }
    hanoi(n - 1, source, destination, auxiliary);
    printf("Move disk %d from %c to %c\n", n, source, destination);
    hanoi(n - 1, auxiliary, source, destination);
}
int main(void) {
    int n;
    printf("Enter number of disks (1-20): ");
    if (scanf("%d", &n) != 1 || n < 1 || n > 20) {
        printf("Enter a number from 1 to 20.\n"); return 1;
    }
    hanoi(n, 'A', 'B', 'C');
    printf("Minimum moves: %llu\n", (1ULL << n) - 1);
    return 0;
}
```

Practical 9: Double-Ended Queue (Deque)

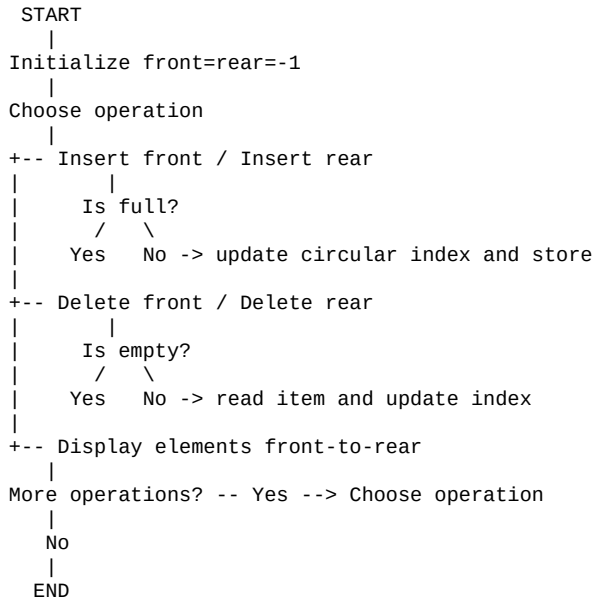
Problem statement

Simulate a deque with operations to add and delete elements from either end.

Algorithm

1. Use a circular array of capacity N, with front and rear indices.
2. Initially set front = rear = -1.
3. Insert at front: reject if full; handle empty deque; otherwise move front backward circularly and store the item.
4. Insert at rear: reject if full; handle empty deque; otherwise move rear forward circularly and store the item.
5. Delete from either end: reject if empty; return the selected value and update indices.
6. Display elements from front to rear circularly.

Flowchart



C Program

```
#include <stdio.h>
#define N 8
int dq[N], front = -1, rear = -1;

int isFull(void) { return front == (rear + 1) % N; }
int isEmpty(void) { return front == -1; }
void insertFront(int x) {
    if (isFull()) { printf("Deque is full.\n"); return; }
    if (isEmpty()) front = rear = 0;
    else front = (front - 1 + N) % N;
    dq[front] = x;
}
void insertRear(int x) {
    if (isFull()) { printf("Deque is full.\n"); return; }
    if (isEmpty()) front = rear = 0;
    else rear = (rear + 1) % N;
    dq[rear] = x;
}
void deleteFront(void) {
    if (isEmpty()) { printf("Deque is empty.\n"); return; }
    printf("Deleted: %d\n", dq[front]);
    if (front == rear) front = rear = -1;
    else front = (front + 1) % N;
}
void deleteRear(void) {
    if (isEmpty()) { printf("Deque is empty.\n"); return; }
    printf("Deleted: %d\n", dq[rear]);
    if (front == rear) front = rear = -1;
    else rear = (rear - 1 + N) % N;
}
void display(void) {
    if (isEmpty()) { printf("Deque is empty.\n"); return; }
    int i = front;
    printf("Deque: ");
    while (1) {
        printf("%d ", dq[i]);
```

```
        if (i == rear) break;
        i = (i + 1) % N;
    }
    printf("\n");
}
int main(void) {
    int ch, x;
    do {
        printf("\n1.Insert front 2.Insert rear 3.Delete front 4.Delete rear 5.Display 0.Exit: ");
        scanf("%d", &ch);
        switch (ch) {
            case 1: printf("Value: "); scanf("%d", &x); insertFront(x); break;
            case 2: printf("Value: "); scanf("%d", &x); insertRear(x); break;
            case 3: deleteFront(); break;
            case 4: deleteRear(); break;
            case 5: display(); break;
            case 0: break;
            default: printf("Invalid choice.\n");
        }
    } while (ch != 0);
    return 0;
}
```

Practical 10: Binary Search Tree Operations

Problem statement

Starting with an empty BST, insert values in the given order, then insert a new node, find the number of nodes on the longest path, find the minimum value, swap left and right pointers at every node, and search for a value.

Algorithm

```
1. Start with root = NULL and insert input values using BST ordering.
2. Insert: smaller values go left; larger values go right. Duplicate values are ignored.
3. Longest path: recursively compute height; height(NULL)=0 and height(node)=1+max(left,right).
4. Minimum: follow left pointers until the leftmost node.
5. Mirror: recursively mirror each subtree, then swap left and right pointers at every node.
6. Search by comparing the key and moving left or right. Note: after mirroring, the tree no longer follows ordinary BST orde
```

Flowchart

```
 START
   |
root = NULL; read values
   |
Insert each value using BST rule
   |
Choose operation
   |
+-- Insert a new value
+-- Height: 1 + max(left height, right height)
+-- Minimum: follow left links
+-- Search: compare key and move left/right
+-- Mirror: recursively swap left and right children
   |
Display requested result / tree
   |
  END
```

C Program

```c
#include <stdio.h>
#include <stdlib.h>

typedef struct Node { int data; struct Node *left, *right; } Node;
Node *newNode(int x) {
    Node *n = malloc(sizeof *n);
    if (!n) { printf("Memory allocation failed.\n"); exit(1); }
    n->data = x; n->left = n->right = NULL; return n;
}
Node *insert(Node *root, int x) {
    if (!root) return newNode(x);
    if (x < root->data) root->left = insert(root->left, x);
    else if (x > root->data) root->right = insert(root->right, x);
    return root;
}
int height(Node *root) {
    if (!root) return 0;
    int l = height(root->left), r = height(root->right);
    return 1 + (l > r ? l : r);
}
Node *minimum(Node *root) {
    if (!root) return NULL;
    while (root->left) root = root->left;
    return root;
}
Node *search(Node *root, int key) {
    while (root && root->data != key)
        root = key < root->data ? root->left : root->right;
    return root;
}
void mirror(Node *root) {
    if (!root) return;
    mirror(root->left); mirror(root->right);
    Node *t = root->left; root->left = root->right; root->right = t;
}
void inorder(Node *root) {
    if (!root) return;
    inorder(root->left); printf("%d ", root->data); inorder(root->right);
}
void freeTree(Node *root) {
    if (!root) return;
    freeTree(root->left); freeTree(root->right); free(root);
}
```

```
int main(void) {
    Node *root = NULL; int n, x, key, ch;
    printf("Number of initial values: "); scanf("%d", &n);
    for (int i = 0; i < n; i++) { scanf("%d", &x); root = insert(root, x); }
    do {
        printf("\n1.Insert 2.Longest path (height) 3.Minimum 4.Search 5.Mirror 6.Inorder 0.Exit: ");
        scanf("%d", &ch);
        switch (ch) {
            case 1: printf("Value: "); scanf("%d", &x); root = insert(root, x); break;
            case 2: printf("Longest path in nodes: %d\n", height(root)); break;
            case 3: { Node *m = minimum(root); if (m) printf("Minimum: %d\n", m->data); else printf("Tree empty.\n"); break;
            case 4: printf("Key: "); scanf("%d", &key);
                    printf(search(root, key) ? "Found.\n" : "Not found.\n"); break;
            case 5: mirror(root); printf("Tree mirrored. BST ordering is now reversed.\n"); break;
            case 6: inorder(root); printf("\n"); break;
            case 0: break;
            default: printf("Invalid choice.\n");
        }
    } while (ch != 0);
    freeTree(root);
    return 0;
}
```

Practical 11: Build a Max Heap

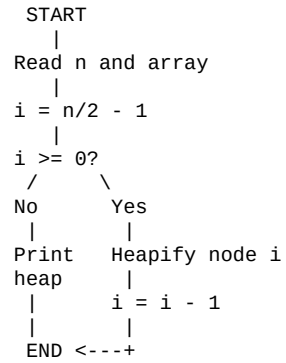
Problem statement

Develop a C program to build a MAX_HEAP with a given input data set. (Content beyond syllabus.)

Algorithm

1. Read n input values into an array.
2. Start heapification from the last non-leaf node, index n/2 - 1.
3. For each node, compare it with its left and right children.
4. Swap with the largest child if the heap property is violated.
5. Continue heapifying the affected subtree.
6. Print the resulting array; every parent is greater than or equal to its children.

Flowchart



C Program

```c
#include <stdio.h>
#define MAX 100

void heapify(int a[], int n, int i) {
    int largest = i, left = 2*i + 1, right = 2*i + 2;
    if (left < n && a[left] > a[largest]) largest = left;
    if (right < n && a[right] > a[largest]) largest = right;
    if (largest != i) {
        int t = a[i]; a[i] = a[largest]; a[largest] = t;
        heapify(a, n, largest);
    }
}
int main(void) {
    int a[MAX], n;
    printf("Enter number of elements (1-100): ");
    if (scanf("%d", &n) != 1 || n < 1 || n > MAX) return 1;
    printf("Enter elements: ");
    for (int i = 0; i < n; i++) scanf("%d", &a[i]);
    for (int i = n/2 - 1; i >= 0; i--) heapify(a, n, i);
    printf("Max heap array: ");
    for (int i = 0; i < n; i++) printf("%d ", a[i]);
    printf("\n");
    return 0;
}
```

Practical 12: Hash Table Using Hashing

Problem statement

Develop a hash table to implement hashing. (Content beyond syllabus.)

Algorithm

1. Create a table of fixed capacity and initialize all slots as empty.
2. Compute the hash index as key modulo table size.
3. Resolve collisions using linear probing: check the next slot cyclically until an empty slot or the same key is found.
4. Insert the key in an empty slot; avoid duplicate keys.
5. Search by following the same probe sequence until the key is found or an empty slot is reached.
6. Display the table and handle a full table safely.

Flowchart

```
 START
   |
Initialize empty hash table
   |
Choose operation
   |
Insert/Search key
   |
index = key % table_size
   |
Probe slots circularly
   |
Key found / empty slot / full cycle?
   |
Insert or report found/not found/full
   |
More operations? -- Yes --> Choose operation
   |
   No
   |
  END
```

C Program

```
#include <stdio.h>
#define SIZE 11
#define EMPTY -1
int table[SIZE];

void init(void) { for (int i = 0; i < SIZE; i++) table[i] = EMPTY; }
int hash(int key) { int h = key % SIZE; return h < 0 ? h + SIZE : h; }
int insert(int key) {
    int start = hash(key);
    for (int step = 0; step < SIZE; step++) {
        int i = (start + step) % SIZE;
        if (table[i] == key) return 1;
        if (table[i] == EMPTY) { table[i] = key; return 1; }
    }
    return 0;
}
int search(int key) {
    int start = hash(key);
    for (int step = 0; step < SIZE; step++) {
        int i = (start + step) % SIZE;
        if (table[i] == EMPTY) return -1;
        if (table[i] == key) return i;
    }
    return -1;
}
void display(void) {
    printf("Index : Key\n");
    for (int i = 0; i < SIZE; i++)
        if (table[i] == EMPTY) printf("%5d : EMPTY\n", i);
        else printf("%5d : %d\n", i, table[i]);
}
int main(void) {
    int ch, key, index;
    init();
    do {
        printf("\n1.Insert 2.Search 3.Display 0.Exit: ");
        scanf("%d", &ch);
        switch (ch) {
            case 1: printf("Key (-1 is reserved as empty): "); scanf("%d", &key);
                    if (key == EMPTY) printf("This key is reserved.\n");
                    else printf(insert(key) ? "Inserted/already present.\n" : "Table full.\n"); break;
```

```
            case 2: printf("Key: "); scanf("%d", &key); index = search(key);
                    if (index >= 0) printf("Found at index %d.\n", index);
                    else printf("Not found.\n"); break;
            case 3: display(); break;
            case 0: break;
            default: printf("Invalid choice.\n");
        }
    } while (ch != 0);
    return 0;
}
```